SMART CONTRACT AUDIT

ZOKYO.

Mar 29, 2021 | v. 1.0

PASS

Zokyo's Security Team has concluded that this smart contract passes security qualifications to be listed on digital asset exchanges.

TECHNICAL SUMMARY

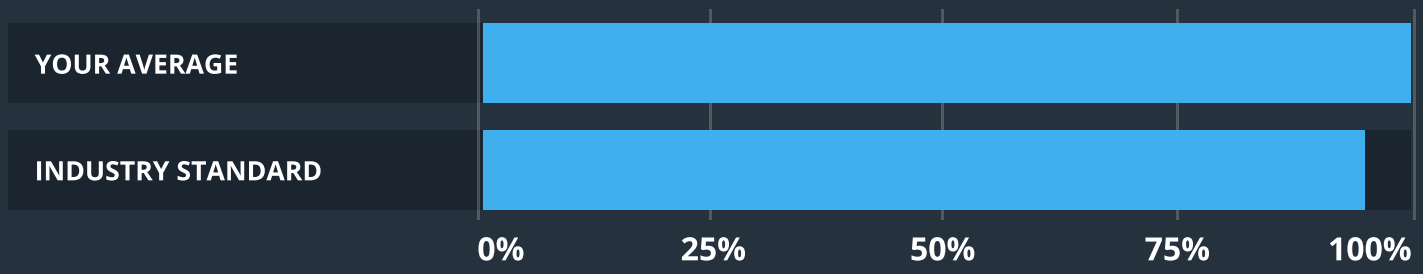
This document outlines the overall security of the Splyt smart contracts ERC20 & Vesting, evaluated by Zokyo's Blockchain Security team.

The scope of this audit was to analyze and document the Splyt smart contract codebase for quality, security, and correctness.

Contract Status

There was one critical issue found during the audit, which was successfully resolved.

Testable Code



Testable code is 100% which is above the industry standard of 95%.

It should be noted that this audit is not an endorsement of the reliability or effectiveness of the contract, rather limited to an assessment of the logic and implementation. In order to ensure a secure contract that's able to withstand the Ethereum network's fast-paced and rapidly changing environment, we at Zokyo recommend that the Splyt team put in place a bug bounty program to encourage further and active analysis of the smart contract.

TABLE OF CONTENTS

Auditing Strategy and Techniques Applied . . . 3
Executive Summary . . . 4
Structure and Organization of Document . . . 5
Manual Review . . . 6
Code Coverage and Test Results for all files . . . 8
Tests written by Zokyo Secured team . . . 8

AUDITING STRATEGY AND TECHNIQUES APPLIED

The Smart contract's source code was taken from the following github repository – https://github.com/splytcore/Splyt_TCPeNFT.
Commit id – d2ed8d7eb0833ba33160c1f8a1bd3b3cd94abf56.

Zokyo team has reviewed only erc20.sol and vesting.sol

Throughout the review process, care was taken to ensure that the token contract:

- Implements and adheres to existing Token standards appropriately and effectively;
- Documentation and code comments match logic and behavior;
- Distributes tokens in a manner that matches calculations;
- Follows best practices in efficient use of gas, without unnecessary waste;
- Uses methods safe from reentrance attacks;
- Is not affected by the latest vulnerabilities;
- Whether the code meets best practices in code readability, etc.

Zokyo's Security Team has followed best practices and industry-standard techniques to verify the implementation of Splyt smart contracts. To do so, the code is reviewed line-by-line by our smart contract developers, documenting any issues as they are discovered. In summary, our strategies consist largely of manual collaboration between multiple team members at each stage of the review:

1	Due diligence in assessing the overall code quality of the codebase.	3	Testing contract logic against common and uncommon attack vectors.
2	Cross-comparison with other, similar smart contracts by industry leaders.	4	Thorough, manual review of the codebase, line-by-line.

EXECUTIVE SUMMARY

There was one critical issue found during the audit, which was successfully resolved. All the mentioned findings may have an effect only in case of specific conditions performed by the contract owner.

Contracts are well written and structured. The findings during the audit have no impact on contract performance or security, so it is fully production-ready.

STRUCTURE AND ORGANIZATION OF DOCUMENT

For ease of navigation, sections are arranged from most critical to least critical. Issues are tagged “Resolved” or “Unresolved” depending on whether they have been fixed or addressed. Furthermore, the severity of each issue is written as assessed by the risk of exploitation or other unexpected or otherwise unsafe behavior:

Critical

The issue affects the ability of the contract to compile or operate in a significant way.

High

The issue affects the ability of the contract to compile or operate in a significant way.

Medium

The issue affects the ability of the contract to operate in a way that doesn’t significantly hinder its behavior.

Low

The issue has minimal impact on the contract’s ability to operate.

Informational

The issue has no impact on the contract’s ability to operate.

MANUAL REVIEW

CRITICAL | RESOLVED

Contract ERC20 lets anyone to mint tokens.

Recommendation:
Minting tokens should be limited to contracts with that right.

HIGH | UNRESOLVED

Contract Vesting does not guarantee existence of tokens to be released on vesting schedule.

Recommendation:
Vesting contract has to transfer tokens to itself to make vesting schedule active.

HIGH | RESOLVED

Smart contracts are not pointing to specific solidity compiler version.

Recommendation:
Point to specific solidity compiler version.

LOW | RESOLVED

Contract Vesting has method name with typo `revocked`.

Recommendation:
Rename method name to `revoked`.

INFORMATIONAL | **UNRESOLVED**

DRY, Smart contract ERC20 is based on openzeppelin library.

Recommendation:
Add openzeppelin to dependencies and inherit openzeppelin ERC20 & ERC20Burnable contracts implementation.

INFORMATIONAL | **RESOLVED**

Smart contract Vesting is not covered by NatSpec annotations.

Recommendation:
Cover by NatSpec all public Contract methods.

CODE COVERAGE AND TEST RESULTS FOR ALL FILES

Tests written by Zokyo Secured team

```
Contract: Vesting
  ✓ should deploy with correct beneficiary (66ms)
  ✓ should deploy with correct cliff period (62ms)
  ✓ should deploy with correct start period (55ms)
  ✓ should deploy with correct  duration period (56ms)
  ✓ should deploy with correct  revocable period (57ms)
  ✓ shouldn't create vesting instance with zero address of beneficiary (178ms)
  ✓ shouldn't cliffDuration be longer than duration of vesting (180ms)
  ✓ shouldn't duration of vesting equal zero (182ms)
  ✓ shouldn't  final time of vesting be less than current time (197ms)
  ✓ should release correct (325ms)
  ✓ shouldn't release when releasable amount is zero (331ms)
  ✓ should revoke correct after cliff period when vesting duration didn't finish (468ms)
  ✓ should revoke correct after 4 months from vesting started (351ms)
  ✓ should revoke correct when vesting duration finished (319ms)
  ✓ should revoke correct when cliff period finished (338ms)
  ✓ shouldn't revoke twice (465ms)
  ✓ shouldn't revoke when vesting is not revocable (429ms)

Contract: ERC20
  ERC20 contract basics
    ✓ should deploy with correct name (60ms)
    ✓ should deploy with correct symbol (54ms)
    ✓ should deploy with expected decimals (46ms)
    ✓ should deploy with expected total supply (80ms)
  ERC20 contract behavior
    ✓ should return expected balance of tokens (97ms)
    ✓ should mint tokens by owner correct (240ms)
    ✓ should approve function work correctl (513ms)
    ✓ should increase allowance correctly (354ms)
    ✓ should decrease allowance correctly (486ms)
    ✓ should transfer tokens with expected behavior (384ms)
    ✓ shouldn't transfer to the zero address (132ms)
```

```
    ✓ shouldn't approve to the zero address (133ms)
    ✓ shouldn't transfer from zero address (142ms)
    ✓ shouldn't mint to zero address (155ms)
    ✓ shouldn't mint by not an owner (170ms)
    ✓ shouldn't mint when cap is exceeded (152ms)

  33 passing (24s)
```

FILE	% STMTS	% BRANCH	% FUNCS	% LINES	UNCOVERED LINES
contracts/	100.00	97.37	100.00	100.00	
ERC20.sol	100.00	94.44	100.00	100.00	
IRC20.sol	100.00	100.00	100.00	100.00	
Vesting.sol	100.00	100.00	100.00	100.00	
All files	**100.00**	**97.37**	**100.00**	**100.00**	

We are grateful to have been given the opportunity to work with the Splyt team.

The statements made in this document should not be interpreted as investment or legal advice, nor should its authors be held accountable for decisions made based on them.

Zokyo's Security Team recommends that the Splyt team put in place a bug bounty program to encourage further analysis of the smart contract by third parties.

ZOKYO.